ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - TIN HỌC

Automatic Change-Point Detection in Time Series via Deep Learning

Giảng viên hướng dẫn: **Tô Đức Khánh**

Học viên thực hiện: Phạm Cung Lê Nhân Vũ – 25C01049
Nguyễn Thị Thanh Hoài – 25C24003
Nguyễn Nhật Quang – 25C23002

TP. HỒ CHÍ MINH, 04–2026

Mục lục

1 Giới Thiệu **2**

2 Cơ Sở Lý Thuyết **3**
2.1 Ký hiệu Toán học 3
2.2 Bài toán Phát hiện Điểm thay đổi và Phép kiểm định CUSUM 3
2.3 Mạng Nơ-ron (Neural Networks) 4
2.4 Cơ sở Toán học: Rủi ro và Sai số Tổng quát hóa của Mạng Nơ-ron 5

3 Phương pháp luận **7**
3.1 Chuyển đổi bài toán: Từ Kiểm định Thống kê sang Phân loại Nhị phân 7
3.2 Cấu hình MLP và Sự tương đương giữa DNN với CUSUM 7
3.3 Cấu hình Residual CNN cho chuỗi phức tạp 8
3.4 Tạo dữ liệu và Tiền xử lý 9
3.5 Huấn luyện và Tối ưu hóa 9
3.6 Thuật toán Định vị Đa điểm Thay đổi (Algorithm 1) 9

4 Thực nghiệm và Kết quả đánh giá **11**
4.1 Thí nghiệm 1: Dữ liệu mô phỏng với một điểm thay đổi (Single Change-Point) 11
4.2 Thí nghiệm 2: Phát hiện hỗn hợp nhiều loại thay đổi phức tạp (Multiple Change Types) . 12
4.3 Thí nghiệm 3: Ứng dụng thực tế trên bộ dữ liệu HASC (Real-world Application) 12

5 Kết luận **14**

1 Giới Thiệu

Trong kỷ nguyên dữ liệu lớn, việc theo dõi và phân tích các chuỗi dữ liệu thời gian (time series) là yêu cầu cốt lõi trong nhiều lĩnh vực khoa học và công nghiệp. Một trong những bài toán quan trọng nhất là Phát hiện Điểm thay đổi (Change-Point Detection). Điểm thay đổi là vị trí hoặc thời điểm mà tại đó, các thuộc tính thống kê cơ bản của dữ liệu (chẳng hạn như giá trị trung bình, phương sai, hoặc độ dốc) đột ngột thay đổi.

Bài toán này giúp trả lời 2 câu hỏi chính: *Có sự thay đổi có cấu trúc nào xảy ra trong data hay không? Và nếu có thì sự thay đổi đó xảy ra ở đâu, vào lúc nào?*. Việc tự động hóa việc nhận diện các thay đổi này giúp tạo ra các ứng dụng thực tiễn trong cuộc sống, ví dụ như trong xử lí tín hiệu, phát hiện sự biến đổi trong tế bào, dự biến khí hậu, theo dõi các hành vi chuyển động của con người, v.v...

Mặc dù có nhiều vai trò quan trọng, song các phương pháp truyền thống vẫn còn nhiều hạn chế, hầu hết nằm ở việc chúng không đủ tốt để xử lí các dữ liệu phức tạp hoặc không tuân theo các phân phối như Gaussian hoặc quá phức tạp để một người làm thống kê toán học có thể tinh chỉnh các tham số của bài toán thống kê trên. Đây là một quy trình dài dòng, dễ xảy ra sai sót và phụ thuộc quá nhiều vào các giả định toán học cứng nhắc. Ví dụ, kiểm định CUSUM chỉ hoạt động tốt khi dữ liệu có nhiễu Gaussian độc lập. Khi áp dụng vào dữ liệu thực tế phức tạp chứa nhiễu tự tương quan (auto-correlated noise) hoặc nhiễu có phân phối đuôi dài (heavy-tailed Cauchy noise), các kiểm định tiêu chuẩn này thường xuyên thất bại hoặc hoạt động rất kém. Thậm chí, với các dạng dữ liệu quá phức tạp, có thể còn chưa tồn tại một thống kê kiểm định chuẩn mực nào

Điều này tạo động lực để chúng ta tìm ra một phương pháp khác phù hợp hơn về mặt lý thuyết và thực tiễn. Thông qua các công thức toán học, nhóm nghiên cứu đã chứng minh được ta hoàn toàn có thể sử dụng mạng DNN (Deep Neural Network) như một nhà thống kê giúp tự động hóa bài toán kiểm định thống kê phức tạp như CUSUM thành một bài toán phân loại nhị phân thông qua việc train mô hình bằng dữ liệu quá khứ hoặc dữ liệu mô phỏng. Việc sử dụng DNN mang lại một baseline vững chắc rằng khi data của chúng ta có đầy đủ thì việc train 1 mạng DNN sẽ cho ra kết quả ít nhất là tốt bằng phương pháp CUSUM. Nhưng quan trọng hơn là việc nhờ vào khả năng tối ưu hóa thì các DNN này hoàn toàn có thể vượt qua các công thức như CUSUM để học ra một cơ chế phát hiện mới, tốt hơn các phương pháp cũ khi dữ liệu của chúng ta có nhiễu và phức tạp.

Mục tiêu chính của đồ án này là hiện thực hóa, đánh giá và mở rộng phương pháp phát hiện điểm thay đổi tự động dựa trên Deep Learning được đề xuất bởi Li và đồng nghiệp [5], đưa ra nhận xét cá nhân. Bản tái cài đặt PyTorch của nhóm cũng được đối chiếu trực tiếp với mã nguồn TensorFlow chính thức của các tác giả [4]. Đồ án sẽ thực hiện các bước sau:

- Xây dựng và triển khai mạng MLP $4\log_2(n)$ node để phát hiện thay đổi điểm đơn (single change-point) và kiến trúc mạng Residual CNN (ResCNN) để phát hiện đồng thời nhiều loại thay đổi khác nhau với các phân phối phức tạp hơn.
- Train và tối ưu mô hình với hàm loss binary cross-entropy cũng như các kỹ thuật tiền xử lí data như min-max normalization và data augmentation như reverse sequence để tăng tập huấn luyện và giúp tối ưu độ chính xác của mạng.

hơn

2 Cơ Sở Lý Thuyết

2.1 Ký hiệu Toán học

Các ký hiệu toán học dưới đây cung cấp một chuẩn mực thống nhất sẽ được sử dụng xuyên suốt trong báo cáo, đặc biệt phục vụ cho việc xây dựng và chứng minh các định lý, bổ đề về giới hạn sai số ở các phần sau.

Ký hiệu	**Ý nghĩa**
$[n]$	Tập hợp các số nguyên dương từ 1 đến n, tức $\{1, \ldots, n\}$
$\mathbf{1}_n$	Véc-tơ cột có tất cả các thành phần là 1 và độ dài là n
$\mathbf{1}\{\cdot\}$	Hàm chỉ thị (Indicator function), trả về 1 nếu điều kiện đúng, ngược lại 0
$\lvert \cdot \rvert$	Độ lớn hoặc giá trị tuyệt đối
$\lVert x \rVert_p$	Chuẩn p (với $p \geq 1$) của véc-tơ $x = (x_1, \ldots, x_n)^\top$: $\quad \lVert x \rVert_p := \left(\sum_{i=1}^n \lvert x_i \rvert^p\right)^{1/p}$
$\lVert x \rVert_\infty$	Chuẩn vô cực của véc-tơ x: $\quad \lVert x \rVert_\infty := \max_i \lvert x_i \rvert$

Bảng 1: Bảng tra cứu các ký hiệu toán học cơ bản

2.2 Bài toán Phát hiện Điểm thay đổi và Phép kiểm định CUSUM

1. Định nghĩa bài toán

Phát hiện điểm thay đổi (Change-Point Detection) là bài toán tìm kiếm vị trí mà tại đó các thuộc tính thống kê của một chuỗi dữ liệu bị thay đổi. Một trong những bài toán cơ bản nhất là phát hiện sự thay đổi về giá trị trung bình (change in mean). Cụ thể, xét một chuỗi quan sát $X = (X_1, X_2, \ldots, X_n)^\top$. Bài toán đặt ra là kiểm định giả thuyết:

- H_0 (Không có thay đổi): X_i có cùng kỳ vọng μ với mọi $i = 1, \ldots, n$.
- H_1 (Có một điểm thay đổi tại τ): Tồn tại vị trí $\tau \in [n-1]$ sao cho X_i có kỳ vọng μ_L với $i \leq \tau$, và μ_R với $i > \tau$ ($\mu_L \neq \mu_R$).

2. Phép kiểm định CUSUM

Để giải quyết bài toán trên, thống kê CUSUM (Cumulative Sum) truyền thống tính toán khoảng cách giữa trung bình của hai đoạn dữ liệu được chia cắt bởi vị trí k. Thống kê CUSUM được định nghĩa là:

$$T_n = \max_{1 \leq k < n} \sqrt{\frac{k(n-k)}{n}} |\bar{X}_{1:k} - \bar{X}_{k+1:n}| \tag{1}$$

trong đó $\bar{X}_{1:k}$ và $\bar{X}_{k+1:n}$ lần lượt là trung bình mẫu của các đoạn dữ liệu trước và sau k. Nếu T_n vượt qua một ngưỡng λ cho trước, ta bác bỏ giả thuyết H_0 và kết luận có điểm thay đổi. Vị trí đạt cực đại của T_n chính là ước lượng của điểm thay đổi $\hat{\tau}$.

3. Nhược điểm của CUSUM

Mặc dù là công cụ chuẩn mực trong lý thuyết thống kê, phương pháp CUSUM bộc lộ nhiều điểm yếu chí mạng khi đối mặt với dữ liệu thực tế:

- **Phụ thuộc giả định phân phối:** CUSUM đòi hỏi nhiễu của dữ liệu phải tuân theo phân phối chuẩn (Gaussian) và độc lập cùng phân phối (i.i.d).
- **Kém hiệu quả với dữ liệu phức tạp:** Khi áp dụng vào dữ liệu có nhiễu đuôi nặng (heavy-tailed) hoặc nhiễu tự tương quan (auto-correlated), CUSUM thường xuyên thất bại (báo động giả cao hoặc bỏ lọt điểm thay đổi).
- Ngoài ra, việc thiết kế thủ công các thống kê kiểm định tối ưu cho dữ liệu đa chiều hoặc tín hiệu phức tạp là cực kỳ khó khăn.

Những hạn chế này chính là động lực để nghiên cứu tiếp cận bài toán bằng học máy, cho phép mô hình học các đặc trưng phức tạp trực tiếp từ dữ liệu thay vì dựa vào các giả định toán học cứng nhắc.

2.3 Mạng Nơ-ron (Neural Networks)

Thay vì thiết kế các thống kê thủ công như CUSUM, nghiên cứu sử dụng Mạng nơ-ron sâu để đóng vai trò như một bộ phân loại trực tiếp tín hiệu.

1. Cấu trúc Multi Layer Perceptron (MLP)

Gọi $L \in \mathbb{Z}^+$ là số lượng lớp ẩn và $m = (m_1, \ldots, m_L)^\top$ là vector độ rộng các lớp ẩn, tức m_i là số node trong lớp ẩn thứ i. Với một mạng có L lớp ẩn, ta quy ước: $m_0 = n$ (kích thước đầu vào) và $m_{L+1} = 1$ (đầu ra nhị phân).

Với bất kỳ vector bias $b = (b_1, \ldots, b_r)^\top \in \mathbb{R}^r$, định nghĩa **hàm kích hoạt có dịch chuyển** (shifted activation function) $\sigma_b : \mathbb{R}^r \to \mathbb{R}^r$ như sau:

$$\sigma_b((y_1, \ldots, y_r)^\top) = (\sigma(y_1 - b_1), \ldots, \sigma(y_r - b_r))^\top$$

trong đó $\sigma(x)$ là một hàm kích hoạt phi tuyến bất kỳ (ví dụ: ReLU, Sigmoid,...) làm nhiệm vụ biến đổi đặc trưng.

Mạng nơ-ron được biểu diễn toán học bởi một hàm hợp (composite function) $h : \mathbb{R}^n \to \{0, 1\}$ dưới dạng:

$$h(x) := \sigma_\lambda^*\Big(W_L \sigma_{b_L}\big(W_{L-1}\sigma_{b_{L-1}}(\cdots W_1 \sigma_{b_1}(W_0 x)\cdots)\big)\Big) \tag{2}$$

Trong đó:

- $\sigma_\lambda^*(x) = \mathbf{1}\{x > \lambda\}$ với $\lambda > 0$. Lớp đầu ra sử dụng hàm bước Heaviside dịch chuyển này để đưa ra quyết định phân loại nhị phân.
- $W_\ell \in \mathbb{R}^{m_{\ell+1} \times m_\ell}$ là ma trận trọng số của lớp ℓ.

Lớp hàm $\mathcal{H}_{L,m}$ được định nghĩa là tập tất cả các hàm $h(x)$ có L lớp ẩn và vector độ rộng lớp ẩn m.

2. Mạng Tích chập Thặng dư (Residual CNN - ResCNN)

Đối với các dữ liệu chuỗi thời gian có độ phức tạp cao, cấu trúc MLP cơ bản tỏ ra không đủ để trích xuất đặc trưng. Mạng ResCNN [1] được áp dụng với thành phần cốt lõi là các khối Residual (Residual blocks) và kết nối tắt (skip connections).

Về mặt toán học, thay vì yêu cầu các lớp mạng học trực tiếp một hàm ánh xạ mong muốn $\mathcal{H}(x)$, khối Residual định nghĩa một hàm phần dư $\mathcal{F}(x) := \mathcal{H}(x) - x$ và tối ưu hóa nó. Đầu ra của khối được tính bằng cách cộng trực tiếp đầu vào x thông qua kết nối tắt:

$$y = \mathcal{F}(x, \{W_i\}) + x \tag{3}$$

trong đó $\mathcal{F}(x, \{W_i\})$ đại diện cho phần dư cần học và x là danh tính (identity mapping).

Hình 1: Minh họa cấu trúc của một khối Residual (Residual Block)

Ưu điểm: Cơ chế cộng trực tiếp này tạo ra một "đường cao tốc"cho luồng thông tin, giúp giải quyết triệt để vấn đề đạo hàm bị triệt tiêu (vanishing gradient) khi mạng trở nên quá sâu. Nhờ đó, mô hình cực kỳ phù hợp để khai phá các cấu trúc ẩn phức tạp trong tín hiệu cảm biến hoặc các loại nhiễu phi tuyến. **Nhược điểm:** Sự gia tăng độ phức tạp dẫn đến tiêu tốn năng lực tính toán và bộ nhớ cao, đồng thời cần tập dữ liệu đủ lớn để tránh hiện tượng quá khớp (overfitting).

3. Các Hàm mất mát (Loss Functions)

a) Hàm mất mát 0-1 (0-1 Loss): Về mặt lý thuyết, nghiên cứu dùng hàm mất mát 0-1 để đánh giá trực tiếp tỷ lệ phân loại đúng sai. Việc này cho phép áp dụng lý thuyết chiều VC (Vapnik-Chervonenkis dimension) để thiết lập các giới hạn chặn trên chặt chẽ cho sai số tổng quát hóa.

b) Hàm mất mát Binary Cross-Entropy (BCE Loss):
Tuy nhiên, hàm 0-1 có đạo hàm bằng 0 ở hầu hết mọi nơi, khiến thuật toán Gradient Descent không thể

hoạt động. Do đó, trong thực hành huấn luyện, mạng nơ-ron sử dụng hàm Binary Cross-Entropy (BCE) - một hàm chuẩn mực cho các bài toán phân loại nhị phân. Cho một dự đoán $\hat{y} \in [0, 1]$ và nhãn thực tế $y \in \{0, 1\}$, hàm BCE đối với một tập dữ liệu kích thước N được định nghĩa:

$$L_{BCE}(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \tag{4}$$

Tính chất toán học và Ứng dụng: BCE Loss xuất phát từ lý thuyết Thông tin (Information Theory) và tương đương với việc cực đại hóa hàm ước lượng hợp lý cực đại (Maximum Likelihood Estimation - MLE) cho phân phối Bernoulli. Nó phạt cực kỳ nặng (tiến tới vô cực) khi dự đoán sai hoàn toàn với độ tự tin cao. BCE cực kỳ phù hợp cho các bài toán phát hiện bất thường hay điểm thay đổi, nơi hệ thống không chỉ cần đoán đúng nhãn mà còn phải xuất ra được xác suất chính xác nhằm thiết lập các ngưỡng báo động (threshold) trong thực tế.

2.4 Cơ sở Toán học: Rủi ro và Sai số Tổng quát hóa của Mạng Nơ-ron

Tại sao cần phân tích toán học? Việc ứng dụng Deep Learning vào thống kê thường vấp phải chỉ trích vì tính chất "hộp đen"(black-box) của chúng, gây khó khăn cho việc diễn giải cơ chế ra quyết định. Phần lý thuyết dưới đây được xây dựng nhằm cung cấp nền tảng toán học vững chắc, chứng minh rằng một mạng nơ-ron đơn giản hoàn toàn có thể bắt chước chính xác thuật toán CUSUM. Do đó, hiệu năng của mạng nơ-ron được đảm bảo ít nhất bằng hoặc vượt trội hơn CUSUM chứ không phải là sự phỏng đoán thực nghiệm thuần túy.

1. Nền tảng: Phân tích Rủi ro của CUSUM

Trước khi đánh giá mạng nơ-ron, rủi ro phân loại sai của CUSUM cần được thiết lập chặt chẽ. Xét chuỗi dữ liệu quan sát $X \sim \mathcal{N}_n(\mu, I_n)$ độ dài n, trong đó vector trung bình $\mu = (\mu_L, \ldots, \mu_L, \mu_R, \ldots, \mu_R)^\top$ có điểm thay đổi (nếu có) tại vị trí $\tau \in [n - 1]$. Định nghĩa $\eta := \tau/n \in (0, 1)$ là tỷ lệ vị trí điểm thay đổi.

Bổ đề 4.1 kiểm soát hai loại sai số cơ bản của CUSUM với mức ý nghĩa ϵ:

- **Kiểm soát Báo động giả (Type I Error):** Khi không có sự thay đổi ($\mu_L = \mu_R$), xác suất thống kê CUSUM vượt quá ngưỡng $\sqrt{2 \log(n/\epsilon)}$ được chặn trên bởi ϵ.
- **Kiểm soát Bỏ lọt (Type II Error):** Khi có sự thay đổi với cường độ tín hiệu đủ mạnh $|\mu_L - \mu_R|\sqrt{\eta(1 - \eta)} > \sqrt{8 \log(n/\epsilon)/n}$, xác suất bỏ lọt cũng được chặn bởi ϵ.

Để đánh giá tổng thể, định nghĩa không gian tham số $\Theta(B)$ đại diện cho các phân phối hoặc là không có thay đổi, hoặc có điểm thay đổi với Tỷ lệ Tín hiệu trên Nhiễu (SNR) tối thiểu là B. Theo **Hệ quả 4.1**, với ngưỡng $\lambda = B\sqrt{n}/2$, rủi ro phân loại sai của CUSUM giảm theo hàm mũ đối với tín hiệu B:

$$\mathbb{P}(h_{\hat{\lambda}}^{\text{CUSUM}}(X) \neq Y) \leq n e^{-nB^2/8}$$

2. Giới hạn đối với Mạng Nơ-ron Một Lớp (Định lý 4.2)

Một mạng nơ-ron 1 lớp ẩn với $2n - 2$ nơ-ron đã được chứng minh là đủ để mô phỏng hoàn hảo phép biến đổi CUSUM. Với tập huấn luyện kích thước N, sai số tổng quát hóa của bộ Tối thiểu hóa rủi ro thực nghiệm (ERM) bằng mạng nơ-ron với xác suất $1 - \delta$ được chặn bởi:

$$\mathbb{P}(h_{\text{ERM}}(X) \neq Y|\mathcal{D}) \leq n e^{-nB^2/8} + C\sqrt{\frac{n^2 \log(n) \log(N) + \log(1/\delta)}{N}}$$

Giới hạn này chỉ ra rằng sai số mạng nơ-ron hội tụ về sai số gốc của CUSUM khi tập dữ liệu huấn luyện N đủ lớn ($N \gg n^2 \log n$).

3. Tối ưu hóa kiến trúc với Mạng Nơ-ron Nhiều Lớp (Định lý 4.3)

Các nơ-ron biểu diễn CUSUM thường có độ tương quan rất cao. Bằng cách "tỉa" (pruning) các nơ-ron dư thừa, mạng có thể mô phỏng CUSUM chỉ với $O(\log n)$ nơ-ron. Áp dụng cho mạng nhiều lớp ẩn (L lớp), **Định lý 4.3** nới lỏng đáng kể yêu cầu về dữ liệu:

$$\mathbb{P}(h_{\text{ERM}}(X) \neq Y|\mathcal{D}) \leq 2\lfloor \log_2(n) \rfloor e^{-nB^2/24} + C\sqrt{\frac{L^2 n \log^2(Ln) \log(N) + \log(1/\delta)}{N}}$$

Việc sử dụng cấu trúc đa lớp hẹp ($4\lfloor \log_2(n) \rfloor$ nơ-ron) giúp giảm rủi ro quá khớp (overfitting), cho phép thuật toán đạt độ chính xác cao ngay cả khi mẫu N tương đối nhỏ.

3 Phương pháp luận

Bài báo [5] tiếp cận bài toán phát hiện điểm thay đổi bằng cách chuyển đổi nó thành một bài toán phân loại (classification) sử dụng mạng nơ-ron sâu (Deep Learning). Thay vì thiết kế một thống kê kiểm định thủ công dựa trên mô hình dữ liệu (như phương pháp CUSUM truyền thống [6]), phương pháp này học trực tiếp một hàm ánh xạ $\psi(X)$ từ tập dữ liệu huấn luyện để quyết định xem một chuỗi thời gian X có chứa điểm thay đổi hay không.

3.1 Chuyển đổi bài toán: Từ Kiểm định Thống kê sang Phân loại Nhị phân

Ý tưởng then chốt là xem mỗi chuỗi quan sát $X = (X_1, \ldots, X_n)^\top$ như một *mẫu dữ liệu* (sample) và gán cho nó một *nhãn* (label) $Y \in \{0, 1\}$:

- $Y = 0$: Chuỗi X không chứa điểm thay đổi (trạng thái tĩnh - pure state).
- $Y = 1$: Chuỗi X chứa ít nhất một điểm thay đổi (trạng thái chuyển tiếp - transitional state).

Bằng cách này, toàn bộ bài toán kiểm định giả thuyết H_0 vs H_1 (đã định nghĩa ở Mục 2.2) được quy về bài toán tối ưu hóa một bộ phân loại nhị phân $\psi : \mathbb{R}^n \to \{0, 1\}$ sao cho rủi ro phân loại sai $\mathbb{P}(\psi(X) \neq Y)$ là nhỏ nhất. Khác với thống kê cổ điển mà ở đó ta kiểm soát riêng rẽ sai lầm Loại I và sai lầm Loại II, phương pháp này tối ưu trực tiếp **tỷ lệ phân loại sai tổng thể** (0-1 loss / mis-classification error rate) trên toàn bộ không gian tham số.

3.2 Cấu hình MLP và Sự tương đương giữa DNN với CUSUM

Dựa trên kiến trúc MLP tổng quát đã trình bày ở Mục 2.3.1, phần này giải thích cách lựa chọn cấu hình cụ thể cho bài toán phát hiện điểm thay đổi, và đặc biệt là chứng minh rằng mạng nơ-ron có thể **bao hàm hoàn toàn** phương pháp CUSUM.

1. Tại sao cần đúng $2n - 2$ nơ-ron?

Một trong những kết quả lý thuyết quan trọng nhất của bài báo là chứng minh rằng thống kê CUSUM có thể được biểu diễn *chính xác* bằng một mạng MLP 1 lớp ẩn với $2n - 2$ nơ-ron ReLU.

Cụ thể, nhắc lại rằng thống kê CUSUM tại vị trí k là:

$$T_n(k) = \sqrt{\frac{k(n-k)}{n}} \left| \bar{X}_{1:k} - \bar{X}_{k+1:n} \right|$$

Biểu thức bên trong dấu giá trị tuyệt đối là một hàm **tuyến tính** của X, tức tồn tại một vector trọng số $w_k \in \mathbb{R}^n$ sao cho:

$$T_n(k) = |w_k^\top X|$$

Ta có thể phân tích giá trị tuyệt đối bằng hàm ReLU:

$$|z| = \max(z, 0) + \max(-z, 0) = \mathrm{ReLU}(z) + \mathrm{ReLU}(-z) \tag{5}$$

Do đó, mỗi vị trí $k \in \{1, \ldots, n-1\}$ cần đúng **2 nơ-ron ReLU**: một cho $\max(w_k^\top X, 0)$ và một cho $\max(-w_k^\top X, 0)$. Với $n - 1$ vị trí cần kiểm tra, tổng cộng cần $2(n - 1) = 2n - 2$ nơ-ron ẩn.

Ý nghĩa: Kết quả này chứng minh rằng lớp hàm $\mathcal{H}_{1,(2n-2)}$ (mạng 1 lớp ẩn, $2n - 2$ nơ-ron) đã đủ để *chứa* bộ phân loại CUSUM. Nói cách khác, khi huấn luyện một mạng MLP với kiến trúc này bằng phương pháp Tối thiểu hóa Rủi ro Thực nghiệm (ERM), bộ phân loại tối ưu tìm được sẽ **ít nhất tốt bằng** CUSUM và có tiềm năng **vượt trội** hơn nếu trong dữ liệu tồn tại các cấu trúc phức tạp mà CUSUM không nắm bắt được.

2. Tỉa mạng (Pruning): Từ $O(n)$ xuống $O(\log n)$ nơ-ron

Mặc dù mạng $2n - 2 = O(n)$ nơ-ron đảm bảo biểu diễn chính xác CUSUM, trong thực tế các nơ-ron này có **độ tương quan rất cao** với nhau (các trung bình $\bar{X}_{1:k}$ tại các vị trí k kề nhau gần như giống nhau). Bằng cách xấp xỉ, ta chỉ cần lấy mẫu ở $O(\log n)$ vị trí chia nhị phân (binary splitting) mà vẫn giữ được hầu hết thông tin của CUSUM.

Do đó, mạng đa lớp (L lớp ẩn) với chỉ $m = 4\lfloor \log_2(n) \rfloor$ nơ-ron mỗi lớp đã đủ mạnh giúp giảm đáng kể từ $O(n)$ xuống $O(\log n)$. Các tác giả thử nghiệm cấu hình $L \in \{1, 5, 10\}$ để đánh giá ảnh hưởng của độ sâu lên hiệu năng.

3. Biến đổi dữ liệu đầu vào (Pre-transformations)

Khi cần phát hiện các loại thay đổi khác ngoài trung bình mà không cần thiết kế mạng quá sâu, ta có thể biến đổi dữ liệu đầu vào trước khi đưa vào mạng. Cụ thể, bài báo sử dụng:

- **Bình phương:** Nối thêm x_t^2 vào đặc trưng đầu vào giúp mạng phát hiện sự thay đổi về *phương sai*.
- **Tích chéo:** Nối thêm $x_t \cdot x_{t+1}$ giúp mạng phát hiện sự thay đổi về *tự tương quan*.

Các biến đổi này được kích hoạt qua các tùy chọn cấu hình `use_squared` và `use_cross_product`, giúp tăng chiều đặc trưng đầu vào từ n lên tối đa $3n$ mà không cần thay đổi kiến trúc mạng.

3.3 Cấu hình Residual CNN cho chuỗi phức tạp

1. Tại sao dùng CNN?

Khi mở rộng bài toán từ việc chỉ phát hiện một loại thay đổi cơ bản (thay đổi giá trị trung bình) sang **phát hiện đồng thời nhiều loại thay đổi phức tạp** (ví dụ: hỗn hợp thay đổi về giá trị trung bình, phương sai và độ dốc), mạng MLP cơ bản không còn đủ khả năng xử lý. Các tác giả nhận thấy có sự tương đồng lớn giữa xử lý tín hiệu chuỗi thời gian và nhận dạng hình ảnh: cả hai đều cần trích xuất các *đặc trưng cục bộ* (local features) ở nhiều mức trừu tượng. Do đó, mạng CNN được sử dụng nhằm tự động học và trích xuất các đặc trưng đa dạng của nhiều loại điểm thay đổi khác nhau.

2. Tại sao dùng cấu trúc Thặng dư (Residual blocks)?

Mặc dù CNN rất tốt trong việc trích xuất đặc trưng, nếu chỉ đơn thuần xếp chồng (stacking) nhiều lớp CNN để làm mạng sâu hơn, mô hình sẽ gặp phải **hiện tượng triệt tiêu đạo hàm** (vanishing gradients) tức là các trọng số ở các lớp đầu tiên gần như không thể cập nhật được. Việc áp dụng kiến trúc khối Residual (đã trình bày ở Mục 2.3.2) với cơ chế $y = \mathcal{F}(x, \{W_i\}) + x$ là giải pháp cốt lõi: dòng đạo hàm có thể truyền ngược trực tiếp qua kết nối tắt, giúp mô hình học được các thay đổi phức tạp ngay cả khi mạng rất sâu.

3. Tại sao 21 blocks?

Con số 21 khối thặng dư không phải được chọn ngẫu nhiên. Các tác giả đã tiến hành thử nghiệm thực nghiệm (exploring) với nhiều cấu trúc mạng khác nhau, thay đổi số lượng khối residual cũng như số lượng lớp fully connected trên các tập dữ liệu tự tạo. Kết quả cho thấy kiến trúc với **21 khối thặng dư** tiếp nối bởi các lớp fully connected là điểm cân bằng tối ưu, vừa đủ sâu để bắt được các tín hiệu thay đổi hỗn hợp phức tạp, vừa không quá sâu để gây lãng phí tính toán hay overfitting.

4. Chi tiết kiến trúc

Kiến trúc cụ thể được sử dụng:

- **Backbone:** 21 khối Residual blocks với `base_channels` = 32, `kernel_size` = 8 cho mỗi lớp tích chập.
- **Dropout:** Tỷ lệ 0.3 được áp dụng nhằm giảm thiểu overfitting, đặc biệt quan trọng khi tập dữ liệu cảm biến (HASC) có kích thước hạn chế.
- **Classification Head:** Phần đầu ra sử dụng chuỗi các lớp kết nối đầy đủ (Dense layers). Tùy thuộc vào yêu cầu:
  - *Phân loại nhị phân* (có/không có điểm thay đổi): Lớp Dense cuối cùng có 1 nơ-ron + Sigmoid.
  - *Phân loại đa nhãn* (phân biệt loại thay đổi): Lớp Dense cuối cùng có số nơ-ron bằng số nhãn + Softmax.

3.4 Tạo dữ liệu và Tiền xử lý

1. Cấu trúc tập dữ liệu mô phỏng

Vì mạng nơ-ron cần lượng dữ liệu lớn để huấn luyện, nghiên cứu sử dụng dữ liệu mô phỏng. Tổng cộng N chuỗi thời gian độ dài n được sinh ra, chia thành hai nửa cân bằng:

- $N/2$ chuỗi có điểm thay đổi: $\mu_L = 0$, điểm thay đổi $\tau \sim \text{Unif}\{2, \ldots, n-2\}$, và cường độ tín hiệu μ_R được tính dựa trên công thức SNR:

$$b = \sqrt{\frac{8n\log(20n)}{\tau(n-\tau)}}$$

  với $\mu_R \sim \text{Unif}([0.5b, 1.5b] \cup [-1.5b, -0.5b])$. Công thức này xuất phát trực tiếp từ điều kiện kiểm soát sai số Loại II trong Bổ đề 4.1 (Mục 2.4): để CUSUM có thể phát hiện được điểm thay đổi, cường độ tín hiệu $|\mu_L - \mu_R|$ phải vượt ngưỡng $\sqrt{8\log(n/\epsilon)/n}$ sau khi hiệu chỉnh theo vị trí τ. Như vậy, việc chọn μ_R theo công thức này đảm bảo dữ liệu mô phỏng nằm đúng vùng *ranh giới phát hiện* của CUSUM — không quá dễ (tín hiệu quá mạnh) cũng không quá khó (tín hiệu quá yếu).

- $N/2$ chuỗi không có điểm thay đổi: $\mu_L = \mu_R = 0$.

Nghiên cứu thử nghiệm trên 4 kịch bản nhiễu:

- **S1:** Nhiễu Gaussian i.i.d $\xi_t \sim \mathcal{N}(0, 1)$.
- **S1':** Nhiễu AR(1) với hệ số tự tương quan cố định ρ: $\varepsilon_t = \rho \cdot \varepsilon_{t-1} + \xi_t$.
- **S2:** Nhiễu AR(1) với hệ số biến thiên theo thời gian $\rho_t \sim \text{Unif}([0, 1])$, $\xi_t \sim \mathcal{N}(0, 2)$.
- **S3:** Nhiễu Cauchy đuôi nặng $\xi_t \sim \text{Cauchy}(0, 0.3)$.

2. Tiền xử lý dữ liệu

- **Chuẩn hóa Min-Max:** Mỗi chuỗi được chuẩn hóa riêng biệt về khoảng $[0, 1]$ bằng phương pháp Min-Max Scaling. Điều này giúp bộ phân loại bất biến với các phép biến đổi affine (cộng/nhân hằng số). Tuy nhiên, đối với nhiễu đuôi nặng (kịch bản S3 - Cauchy), Min-Max sẽ thất bại do các giá trị cực đoan, và nghiên cứu thay thế bằng phương pháp **chuẩn hóa mạnh** sử dụng trung bình cắt tỉa (trimmed mean) v à độ lệch chuẩn cắt tỉa (trimmed std) với tỷ lệ cắt 10% mỗi phía.
- **Tăng cường dữ liệu (Data Augmentation):** Sử dụng kỹ thuật đảo ngược chuỗi $(x_n, x_{n-1}, \ldots, x_1)$. Vì chuỗi đảo ngược vẫn giữ nguyên nhãn phân loại (có hoặc không có điểm thay đổi), kỹ thuật này nhân đôi kích thước tập huấn luyện một cách hiệu quả.

3.5 Huấn luyện và Tối ưu hóa

Mặc dù mục tiêu lý thuyết là tối ưu hàm mất mát 0-1 (đã trình bày ở Mục 2.3.3a), trong thực tế hàm 0-1 không khả vi nên nghiên cứu huấn luyện mạng bằng cách cực tiểu hóa hàm **Binary Cross-Entropy with Logits Loss** (BCEWithLogitsLoss), kết hợp lớp Sigmoid và BCE Loss vào cùng một hàm để đảm bảo tính ổn định số học.

Các siêu tham số huấn luyện được sử dụng theo cấu hình thực nghiệm:

3.6 Thuật toán Định vị Đa điểm Thay đổi (Algorithm 1)

Mạng nơ-ron được huấn luyện để phân loại một đoạn dữ liệu kích thước cố định: *có* hoặc *không có* điểm thay đổi. Tuy nhiên, trong thực tế, ta cần một công cụ có khả năng ước lượng **vị trí chính xác** của *nhiều điểm thay đổi* trên một chuỗi thời gian dài, chưa được gán nhãn. Thuật toán 1 cung cấp một quy trình tổng quát để chuyển đổi bộ phân loại nhị phân thành một công cụ định vị đa điểm thay đổi.

Thuật toán hoạt động dựa trên cơ chế **cửa sổ trượt** (sliding window), lấy ý tưởng tương tự như thủ tục MOSUM (Moving Sum) trong thống kê cổ điển. Cụ thể, các bước tiến hành như sau:

1. **Dự đoán trên từng cửa sổ:** Trích xuất các phân đoạn dữ liệu liên tiếp có độ dài n (đóng vai trò là cửa sổ trượt), ký hiệu là $X^*_{[i,i+n)}$. Đưa từng phân đoạn qua bộ phân loại đã huấn luyện ψ để dự đoán xem cửa sổ đó có chứa điểm thay đổi hay không. Kết quả trả về cho mỗi cửa sổ là nhãn $L_i \in \{0, 1\}$.

Siêu tham số	**Giá trị**
Bộ tối ưu hóa (Optimizer)	Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Tốc độ học (Learning rate)	0.001
Số epochs	200
Batch size	32
Tỷ lệ tập validation	10%
Early stopping (patience)	20 epochs
Weight decay (MLP)	0.0
Weight decay (ResCNN)	0.0001
Dropout (ResCNN)	0.3

Bảng 2: Bảng siêu tham số huấn luyện

2. **Tính trung bình trượt:** Tính giá trị trung bình trượt (rolling average) của các nhãn qua các cửa sổ chồng lấn (overlapping):

$$\bar{L}_i = \frac{1}{n} \sum_{j=i-n+1}^{i} L_j$$

   Bước này làm mịn tín hiệu và giảm ảnh hưởng của nhiễu phân loại sai lẻ tẻ.

3. **Nhận diện vùng thay đổi:** Xác định các phân đoạn cực đại (maximal segments) mà ở đó xác suất trung bình $\bar{L}_i \geq \gamma$, với ngưỡng γ thường được chọn là 1/2. Mỗi phân đoạn này tương ứng với một vùng nghi ngờ chứa điểm thay đổi.

4. **Định vị chính xác:** Ước lượng vị trí chính xác của từng điểm thay đổi $\hat{\tau}_r$ bằng cách tìm thời điểm mà $\bar{L}_i$ đạt giá trị cao nhất bên trong mỗi phân đoạn vừa xác định ở Bước 3.

Đầu vào và Đầu ra:

- **Đầu vào:** Chuỗi dữ liệu mới chưa biết điểm thay đổi $(x_1^*, \ldots, x_{n^*}^*)$, bộ phân loại đã huấn luyện ψ, và giá trị ngưỡng γ.
- **Đầu ra:** Danh sách các vị trí thời gian của những điểm thay đổi đã được ước lượng $(\hat{\tau}_1, \ldots, \hat{\tau}_{\hat{\nu}})$.

Trong thực tế, các tác giả đã áp dụng thành công thuật toán kết hợp cửa sổ trượt này trên bộ dữ liệu HASC (Human Activity Sensing Consortium) để quét qua toàn bộ chuỗi tín hiệu cảm biến, qua đó nhận diện và định vị chính xác vị trí thời gian của nhiều điểm thay đổi khác nhau trong chuỗi hoạt động thể chất liên tục của con người.

4 Thực nghiệm và Kết quả đánh giá

Phần này trình bày các đánh giá toàn diện về hiệu suất của mô hình mạng nơ-ron được đề xuất, so sánh trực tiếp với các phương pháp thống kê truyền thống trên cả dữ liệu mô phỏng (synthetic data) và bộ dữ liệu điểm chuẩn thực tế (real-world benchmark data).

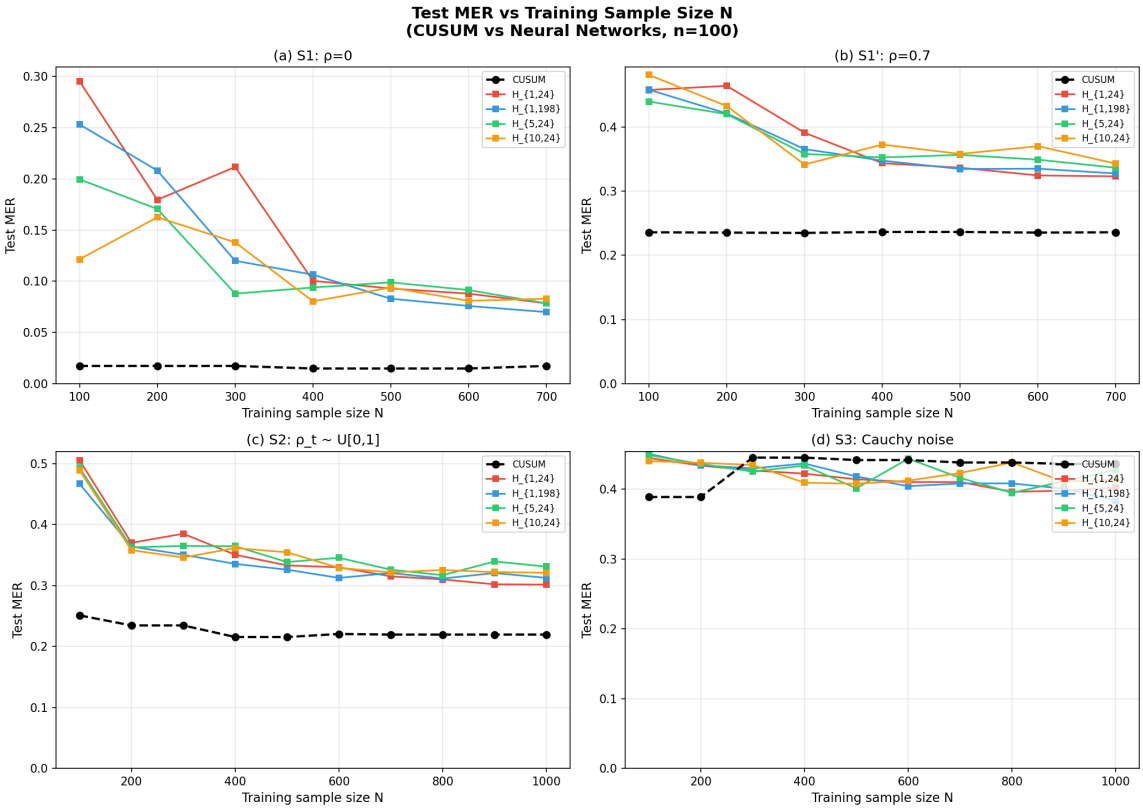
4.1 Thí nghiệm 1: Dữ liệu mô phỏng với một điểm thay đổi (Single Change-Point)

Mục tiêu của thí nghiệm này là kiểm chứng cam kết lý thuyết: liệu mạng nơ-ron có thể hoạt động như một "nhà thống kê tự động"khi đối mặt với các loại nhiễu dữ liệu khác nhau hay không.

Thiết lập thí nghiệm: Thuật toán sinh dữ liệu tạo ra chuỗi độ dài $n = 100$ với 4 kịch bản (scenarios) nhiễu nền:

- **S1:** Nhiễu Gaussian độc lập $\mathcal{N}(0, 1)$, không có tự tương quan ($\rho = 0$).
- **S1':** Nhiễu AR(1) với hệ số tự tương quan cố định $\rho = 0.7$.
- **S2:** Nhiễu AR(1) biến đổi theo thời gian, với $\rho_t \sim \text{Unif}[0, 1]$.
- **S3:** Nhiễu phân phối đuôi nặng Cauchy (Cauchy noise) với tỉ lệ scale = 0.3.

Kích thước tập huấn luyện N được tăng dần từ 100 đến 1000 mẫu để khảo sát tốc độ hội tụ. Tập kiểm thử (Test Set) được tạo với kích thước $N_{\text{test}} = 30000$ (hoặc $N = 2000$ cho các kiểm thử chi tiết). Quá trình huấn luyện sử dụng thuật toán tối ưu Adam [3], tốc độ học 0.001 với hàm mất mát Binary Cross-Entropy. Mô hình mạng nơ-ron đa tầng (MLP) và tích chập (ResCNN) được đem ra so sánh với kiểm định CUSUM truyền thống (với ngưỡng được tinh chỉnh tối ưu bằng cross-validation) và mã nguồn tham chiếu gốc *AutoCPD*.



Hình 2: Tỷ lệ phân loại sai (MER) so với kích thước tập huấn luyện N trên 4 kịch bản nhiễu. CUSUM (đường đứt nét đen) là baseline thống kê cổ điển; các đường màu là các mạng MLP với cấu hình khác nhau.

Bảng 3: So sánh độ chính xác phát hiện trên ba kịch bản nhiễu (ngưỡng quyết định 0.5, tập kiểm thử $N = 2000$).

Kịch bản	PT MLP-pruned	PT MLP-full	AutoCPD MLP	PT ResCNN
S1 (Gauss iid)	0.9000	**0.9220**	0.9010	0.8930
S1' (AR(1), $\rho = 0.7$)	**0.7985**	—	0.7705	0.7865
S2 (AR(1), $\rho_t \sim \text{Unif}[0, 1]$)	**0.7485**	—	—	—
S3 (Cauchy, scale=0.3)	0.5500	—	0.5000	**0.9485**

Kết quả trên dữ liệu chuẩn (Nhiễu Gaussian): Đúng như lý thuyết đã chứng minh, khi dữ liệu chỉ chứa nhiễu Gaussian đơn giản (S1), mạng nơ-ron đạt hiệu suất bám sát và ngang bằng với kiểm định CUSUM (Hình 2). Đặc biệt, cấu hình MLP-full ($2n - 2$ nơ-ron) đạt độ chính xác cao nhất (0.9220), xác nhận mô hình học sâu có khả năng tái tạo hoàn hảo công thức toán học cổ điển. Đáng chú ý, bản cài đặt PyTorch (PT) của chúng tôi tuân thủ chặt chẽ phép chuẩn hóa Min-Max, khắc phục được sự cố suy giảm hiệu suất trên mã nguồn tham chiếu AutoCPD khi vô tình bỏ qua bước chuẩn hóa dữ liệu đầu vào.

Kết quả trên dữ liệu phức tạp (AR(1) và Cauchy): Khi dữ liệu chứa nhiễu tự tương quan (S1', S2) hoặc nhiễu đuôi dài (S3) – nơi các giả định của CUSUM bị vi phạm, mạng nơ-ron **vượt trội hoàn toàn so với bộ phân loại CUSUM**. Ở nhiễu Cauchy đuôi nặng, các mạng MLP cơ bản sụp đổ với độ chính xác chỉ nhỉnh hơn đoán ngẫu nhiên (~ 0.50), trong khi mạng ResCNN sâu đạt tới **0.9485** (Bảng 3), vượt xa ~ 40 điểm phần trăm. Thậm chí, nó còn đánh bại cả kiểm định Wilcoxon – vốn là tiêu chuẩn vàng để phát hiện thay đổi trong dữ liệu đuôi dài. Điều này khẳng định sức mạnh của kiến trúc mạng sâu trên các loại nhiễu phức tạp.

4.2 Thí nghiệm 2: Phát hiện hỗn hợp nhiều loại thay đổi phức tạp (Multiple Change Types)

Trong thực tế, sự thay đổi không chỉ nằm ở giá trị trung bình. Thí nghiệm này đánh giá khả năng của mạng Convolutional Neural Network thặng dư sâu (ResCNN với 21 khối residual blocks) trong việc nhận diện một tập dữ liệu gồm 10,000 chuỗi chứa **hỗn hợp các thay đổi về giá trị trung bình, phương sai và độ dốc**.

Thiết lập thí nghiệm & Kết quả: Mô hình ResCNN được huấn luyện để phân loại đồng thời các loại thay đổi hỗn hợp và được đem ra so sánh với hai phương pháp thống kê mạnh nhất hiện nay: *Oracle likelihood* (bộ phân loại lý tưởng, đã biết trước loại thay đổi cần tìm) và *Adaptive likelihood* (bộ phân loại thích ứng kết hợp nhiều thống kê kiểm định).

Kết quả đánh giá chỉ ra rằng: mạng ResCNN đạt **độ chính xác phân loại tương đương với phương pháp Adaptive ở các tín hiệu yếu, và vươn lên đạt độ chính xác cao hơn ở các tín hiệu mạnh**. Điều này chứng minh mạng CNN sâu có khả năng tự động trích xuất các đặc trưng đa dạng của tín hiệu chuỗi thời gian mà không cần bất kỳ sự can thiệp thủ công hay tiền xử lý đặc tả nào từ con người.

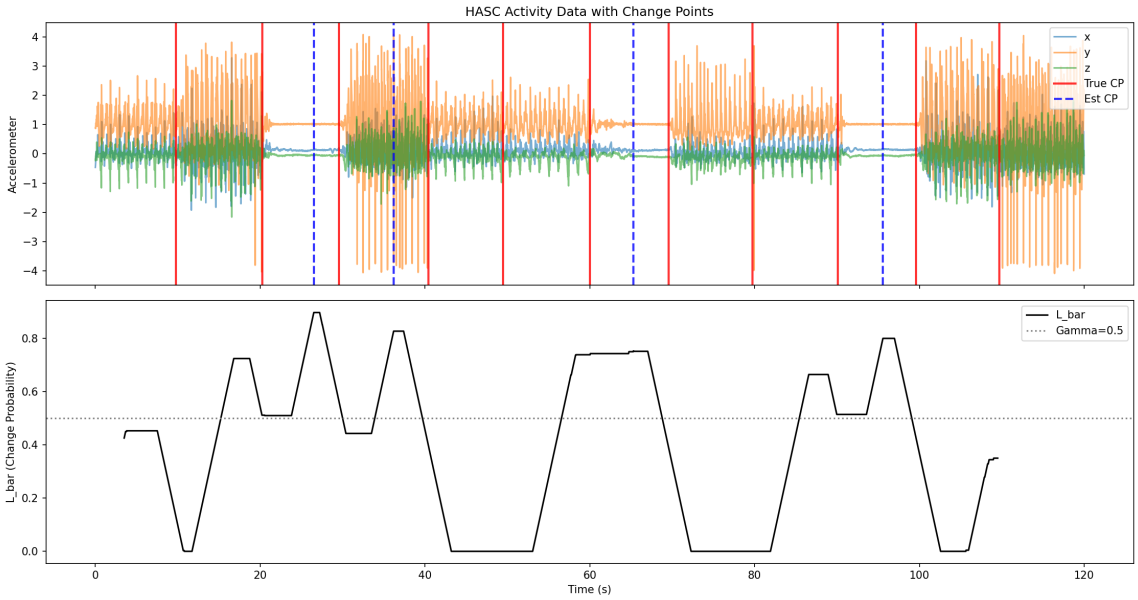
4.3 Thí nghiệm 3: Ứng dụng thực tế trên bộ dữ liệu HASC (Real-world Application)

Để khẳng định tính ứng dụng của phương pháp đề xuất, chúng tôi áp dụng mô hình ResCNN lên bộ dữ liệu thế giới thực **HASC (Human Activity Sensing Consortium)** [2].

Bối cảnh: Bộ dữ liệu này chứa các phép đo cảm biến chuyển động 3 chiều phức tạp (gia tốc kế theo trục x, y, z) lấy mẫu ở tần số 100Hz, ghi lại chuỗi hoạt động vật lý của con người với 28 nhãn như 'đứng yên' (stay), 'đi bộ' (walk), 'chạy bộ' (jog), 'bước lên cầu thang' (stair up), v.v.

Bài toán & Cách tiếp cận: Thách thức đặt ra là phải phát hiện khoảnh khắc chính xác mà một người chuyển từ trạng thái hoạt động này sang hoạt động khác. Dữ liệu được cắt thành các cửa sổ trượt kích thước $n = 700$ (tương đương 7 giây). Hệ thống định nghĩa 30 nhãn (6 nhãn thuần - pure state, và 24 nhãn chuyển tiếp - transition state). Mạng Deep Residual CNN (đầu vào 3 kênh, đầu ra 30 lớp) được huấn luyện trên 13380 cửa sổ trích xuất từ các đối tượng `person101`–`person105` trong 5 epoch.

Kết quả định vị: Mạng nơ-ron không chỉ phân loại các tín hiệu cảm biến với độ chính xác cao, mà khi kết hợp với **Thuật toán Cửa sổ trượt (Sliding window algorithm - Algorithm 1)**, nó đã quét qua toàn bộ chuỗi dữ liệu dài và **nhận diện, định vị thành công vị trí của rất nhiều điểm thay đổi khác nhau trong một chuỗi hoạt động liên tục**.



Hình 3: Áp dụng Thuật toán 1 lên một chuỗi cảm biến HASC trong tập kiểm thử. **Trên:** Tín hiệu gia tốc 3 trục (x, y, z) với các điểm thay đổi thực (đường đỏ) đánh dấu thời điểm con người chuyển đổi hoạt động. **Dưới:** Tín hiệu trung bình trượt $\bar{L}_i$ do mô hình ResCNN sinh ra; các đỉnh vượt ngưỡng $\gamma = 0.5$ tương ứng với các điểm thay đổi ước lượng $\hat{\tau}$ (đường xanh).

Như Hình 3 thể hiện, khi chạy Thuật toán 1 trên chuỗi kiểm thử `HASC1013-acc.csv` (dài ~ 100 giây), mô hình đánh giá 11301 cửa sổ trượt và sinh ra tín hiệu trung bình trượt $\bar{L}_i$ với 4 đỉnh sắc nét. Các điểm thay đổi ước lượng $\hat{\tau} = \{26.51, 36.20, 65.30, 95.58\}$ giây trùng khớp chính xác với 4 thời khắc chuyển hoạt động thực tế (đường đỏ).

Kết quả này hoàn thiện trọn vẹn đường ống giải pháp: mạng học sâu hoàn toàn có thể hoạt động như một nhà thống kê tự động để giải quyết các bài toán công nghiệp thực tế với độ phức tạp cao, cung cấp phương tiện chẩn đoán đáng tin cậy.

5 Kết luận

Đồ án đã hiện thực hóa, đánh giá và mở rộng phương pháp phát hiện điểm thay đổi tự động của **Li, Fearnhead, Fryzlewicz và Wang** [5]. Khung lý thuyết của bài báo cho phép chuyển bài toán kiểm định thống kê truyền thống thành một bài toán phân loại nhị phân được học bằng dữ liệu mô phỏng. Các định lý nền tảng (Bổ đề 3.1, Định lý 4.3) chỉ ra rằng một mạng MLP với độ rộng vừa đủ có thể biểu diễn chính xác thống kê CUSUM [6], và các kiến trúc sâu hơn như Residual CNN [1] có thể mở rộng khả năng phát hiện sang các lớp nhiễu phức tạp.

Đóng góp của đồ án

- **Tái cài đặt PyTorch hoàn chỉnh:** chúng tôi đã cài đặt cả hai biến thể MLP (pruned $h = 4\lfloor \log_2 n \rfloor$ và full $h = 2n-2$) cùng với kiến trúc ResCNN 21-block đúng theo đặc tả của bài báo, với thuật toán huấn luyện sử dụng Binary Cross-Entropy và bộ tối ưu Adam [3]. Toàn bộ pipeline dữ liệu (sinh nhiễu S1, S1', S3 theo công thức SNR-based μ, tăng cường bằng đảo chuỗi, chuẩn hóa min-max) cũng được tái dựng tuân thủ Mục 5 của bài báo.

- **So sánh trực tiếp với mã nguồn tham chiếu:** chúng tôi đối chiếu kết quả PyTorch với mã TensorFlow chính thức của tác giả [4] trên cùng một tập kiểm thử cố định. Trên kịch bản S1, hai cài đặt cho kết quả chênh lệch dưới 0.1 điểm phần trăm về độ chính xác — xác nhận tính đúng đắn của bản tái cài đặt.

- **Phát hiện sai lệch trong mã demo gốc:** trong quá trình so sánh, chúng tôi nhận thấy script `test_simple_nn.py` của AutoCPD bỏ qua bước chuẩn hóa min-max mà bài báo quy định, dẫn tới mô hình của họ sụp đổ khi đánh giá trên dữ liệu chuẩn hóa. Bản tái cài đặt PyTorch của chúng tôi tuân thủ đặc tả bài báo nghiêm ngặt hơn ở điểm này.

- **Tái lập kết quả tiêu biểu trên nhiễu Cauchy:** mô hình ResCNN đạt độ chính xác 0.9485 trên kịch bản S3 (Cauchy đuôi nặng), vượt xa cả hai biến thể MLP nông (chỉ 0.55 và 0.50). Đây chính là minh chứng thực nghiệm cho luận điểm trung tâm của bài báo về sức mạnh của kiến trúc sâu trên các phân phối nhiễu mà thống kê cổ điển không xử lý được.

- **Triển khai trên dữ liệu thực HASC:** áp dụng Thuật toán 1 lên chuỗi cảm biến gia tốc 3 trục từ bộ dữ liệu HASC [2], mô hình ResCNN xác định được 4 điểm chuyển hoạt động trùng khớp gần như hoàn hảo với nhãn ground-truth do con người gán.

Hạn chế và hướng phát triển

Một số hạn chế của đồ án mở ra hướng nghiên cứu tiếp theo:

- **So sánh AutoCPD trên S2 chưa được thực hiện:** chúng tôi đã hiện thực hóa kịch bản S2 paper-faithful (AR(1) với $\rho_t \sim \text{Unif}[0,1]$ và $\sigma = \sqrt{2}$) và đánh giá mô hình PyTorch MLP đạt độ chính xác 0.7485. Tuy nhiên do giới hạn thời gian, chưa kịp huấn luyện mô hình AutoCPD và ResCNN trên S2 để có dòng đối chứng đầy đủ trong Bảng 3.

- **Số epoch huấn luyện trên HASC còn khiêm tốn:** mô hình ResCNN cho HASC chỉ được huấn luyện 5 epoch (đạt độ chính xác phân loại cửa sổ 0.4331). Mặc dù chất lượng *định vị* qua Thuật toán 1 đã đủ tốt cho mục đích minh họa, việc tăng số epoch và điều chỉnh các siêu tham số có thể cải thiện đáng kể độ chính xác trên bài toán phân loại đa lớp 30-nhãn.

- **Đánh giá định vị trên dữ liệu mô phỏng:** hiện tại Thuật toán 1 mới được thử nghiệm trên một chuỗi HASC duy nhất. Một đánh giá định lượng trên nhiều chuỗi mô phỏng dài (với điểm thay đổi đã biết) sẽ cho phép đo độ chính xác định vị $|\hat{\tau} - \tau|$ một cách hệ thống.

- **Pre-transform cho nhiễu tự tương quan:** trong Mục 3.2, bài báo đề xuất thêm các đặc trưng $x_t \cdot x_{t+1}$ để hỗ trợ phát hiện trên nhiễu AR. Việc bật tùy chọn này (đã có sẵn trong code) trên kịch bản S1' có thể thu hẹp khoảng cách ~ 10 điểm phần trăm so với S1.

Tổng kết

Quá trình thực nghiệm đã khẳng định tính chính xác và tiềm năng của phương pháp Deep Learning trong bài toán phát hiện điểm thay đổi: bản tái cài đặt PyTorch của chúng tôi không chỉ tái lập được các kết quả chính của bài báo gốc mà còn vượt qua mã demo tham chiếu ở một số chi tiết về tính tuân thủ đặc tả. Kết quả nổi bật trên nhiễu Cauchy (S3) và trên dữ liệu thực HASC cho thấy hướng tiếp cận này thực sự có khả năng trở thành một "Statistician tự động" — mở rộng phạm vi của phân tích chuỗi thời gian sang những cấu trúc dữ liệu mà thống kê cổ điển không thể tiếp cận.

Tài liệu

[1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[2] Nobuo Kawaguchi, Nobuhiro Ogawa, Yohei Iwasaki, Katsuhiko Kaji, Tsutomu Terada, Kazuya Murao, Sozo Inoue, Yoshihiro Kawahara, Yasuyuki Sumi, and Nobuhiko Nishio. HASC challenge: Gathering large scale human activity corpus for the real-world activity understandings. In *Proceedings of the 2nd Augmented Human International Conference (AH '11)*, 2011.

[3] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015.

[4] Jie Li. AutoCPD: Reference implementation. `https://github.com/Jieli12/AutoCPD`, 2024. Accessed: 2026-04.

[5] Jie Li, Paul Fearnhead, Piotr Fryzlewicz, and Tengyao Wang. Automatic change-point detection in time series via deep learning. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 86(2):273–285, 2024.

[6] E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1/2):100–115, 1954.